Projektuppgift

DT224G – Introduktion till webbutveckling med HTML, CSS och JavaScript
Respawn-After-Bedtime
Utveckling av en responsiv webbplats för en spelgemenskap

Mikael Emanuelsson 2026



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

Respawn after bedtime
Mikael Emanuelsson 2026-10-27

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Mikael Emanuelsson, miem2600@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Sammanfattning

Projektets syfte var att utveckla en responsiv webbplats för *Respawn After Bedtime*, en fiktiv spelgemenskap riktad till vuxna över 30 år. Webbplatsen ska göra det enkelt för besökare att hitta information om gemenskapen, upptäcka spel och ansluta till dess Discord-server.

Arbetet inleddes med planering och design i Figma, där wireframes och en grafisk utformning togs fram för mobil och dator. Webbplatsen utvecklades därefter med HTML, CSS och JavaScript enligt principen Mobile First. CSS Grid och Flexbox användes för att anpassa layouten till olika skärmstorlekar. JavaScript används bland annat för mobilmenyn, validering av kontaktformuläret och dynamisk visning av spelinformation.

Spelinformation hämtas från RAWG API genom ett separat Node.js-skript och sparas i en lokal JSON-fil. Webbplatsen läser sedan filen med JavaScript. På så sätt kan informationen publiceras utan att den privata API-nyckeln behöver finnas i webbplatsens publika kod.

Resultatet är en publicerad webbplats med fem sammanlänkade sidor. Webbplatsen har testats med avseende på responsivitet, funktionalitet, kodvalidering och tillgänglighet. Projektet visar hur HTML, CSS och JavaScript kan kombineras för att skapa en användbar och responsiv webbplats, samtidigt som externa data hanteras utan att känsliga uppgifter exponeras.

Innehåll

Sammanfattning 2
1 Introduktion 4
1.1 Bakgrund och problemmotivering 4
1.2 Övergripande syfte 4
1.3 Avgränsningar 4
1.4 Konkreta och verifierbara mål 5
1.5 Översikt 5
2 Teoretisk bakgrund 6
2.1 HTML, CSS och JavaScript 6
2.2 Responsiv webbdesign och Mobile First 6
2.3 DOM och händelsehantering 7
2.4 API och JSON 7
2.5 Tillgänglighet och webbstandarder 7
3 Metod 9
3.1 Planering och design 9
3.2 Utvecklingsmiljö och versionshantering 11
3.3 Arbetsprocess och problemlösning 11
3.4 Testning och kvalitetssäkring 11
4 Konstruktion 13
4.1 Webbplatsens struktur 13
4.2 Responsiv layout och navigation 13
4.3 JavaScript och interaktivitet 14
4.4 RAWG API och hantering av speldata 15
5 Resultat 17
5.1 Måluppfyllelse 17
5.2 Testresultat 17
6 Diskussion och slutsats 19
7 Källförteckning 20

1 Introduktion

Respawn After Bedtime är ett fiktivt community för vuxna spelare över 30 år. Inom projektet har en responsiv webbplats utvecklats för att presentera gemenskapen, visa populära spel och göra det enkelt för besökare att ansluta via Discord.

Projektet har genomförts inom kursen Introduktion till webbutveckling med HTML, CSS och JavaScript vid Mittuniversitetet. Arbetet omfattar planering, design, utveckling, testning och publicering av webbplatsen.

1.1 Bakgrund och problemmotivering

För vuxna med intresse för datorspel kan det vara svårt att hitta andra att spela tillsammans med när arbete, familj och andra vardagliga åtaganden begränsar tiden för spelande. Respawn After Bedtime bygger på idén om en gemenskap där vuxna spelare kan umgås och spela tillsammans utan krav på regelbunden närvaro.

Målgruppen är personer över 30 år som söker en avslappnad spelgemenskap. Webbplatsen ska fungera som en samlad informationsplats där besökare kan läsa om gemenskapen, upptäcka spel och hitta vidare till dess Discord-server.

För att webbplatsen ska vara enkel att använda oavsett enhet behöver innehållet vara tydligt strukturerat och anpassat för både mobiltelefoner och större skärmar.

1.2 Övergripande syfte

Projektets övergripande syfte är att utveckla en responsiv och användarvänlig webbplats som presenterar Respawn After Bedtime och gör det enkelt för målgruppen att hitta information om spelgemenskapen och ansluta till den.

Projektet syftar även till att omsätta kunskaper från kursen i praktiken genom att kombinera HTML, CSS och JavaScript i en sammanhängande webbplats. Särskild vikt läggs vid responsiv design, tydlig navigation och tillgänglighet.

1.3 Avgränsningar

Projektet är avgränsat till utvecklingen av en responsiv webbplats med HTML, CSS och JavaScript. Webbplatsen består av fem sidor och fokuserar på att presentera spelgemenskapen, visa spelinformation och hänvisa besökare till Discord.

Ingen egen användarhantering, databas eller serverbaserad funktionalitet har utvecklats. Kontaktformuläret demonstrerar validering med JavaScript men skickar inte några meddelanden.

Spelinformation hämtas från RAWG API genom ett separat skript och sparas i en lokal JSON-fil. Webbplatsen gör därför inga direkta API-anrop från besökarens webbläsare. Denna lösning valdes för att hålla API-nyckeln utanför den publika koden.

1.4 Konkreta och verifierbara mål

För att uppfylla projektets syfte formulerades följande mål:

1. Utveckla en webbplats med minst tre sammanlänkade HTML-sidor som presenterar verksamheten och dess innehåll.
2. Skapa en responsiv layout enligt principen Mobile First, med CSS Grid, Flexbox och media queries.
3. Implementera en mobilmeny och ett kontaktformulär med JavaScript-baserad validering.
4. Visa spelinformation dynamiskt genom att läsa data från en lokal JSON-fil med JavaScript.
5. Säkerställa att webbplatsen fungerar på olika skärmstorlekar och kan användas med tangentbord.
6. Validera HTML och CSS samt genomföra tillgänglighetstester.
7. Publicera webbplatsen via GitHub Pages och dokumentera projektet i ett GitHub-repository.

Måluppfyllelsen redovisas i kapitel 5, Resultat, utifrån den färdiga webbplatsen och genomförda tester.

1.5 Översikt

Kapitel 2 behandlar den tekniska bakgrunden och de tekniker som används i projektet. Kapitel 3 beskriver arbetsmetod, planering och verktyg. I kapitel 4 redovisas webbplatsens konstruktion och de viktigaste tekniska lösningarna. Kapitel 5 presenterar projektets resultat och genomförda tester. Slutligen diskuteras arbetets genomförande, begränsningar och möjliga förbättringar i kapitel 6.

2 Teoretisk bakgrund

Webbplatsen Respawn After Bedtime bygger på HTML, CSS och JavaScript. Dessa tekniker används tillsammans för att strukturera innehåll, utforma webbplatsens utseende och skapa interaktiva funktioner.

I detta kapitel beskrivs de grundläggande tekniker och principer som är relevanta för projektet, med fokus på responsiv webbdesign, JavaScript, hantering av externa data och tillgänglighet. Den teoretiska bakgrunden utgår från kursmaterial, kurslitteratur och webbdokumentation.

2.1 HTML, CSS och JavaScript

HTML (HyperText Markup Language) används för att strukturera innehållet på en webbsida med hjälp av element som rubriker, stycken, länkar och formulär. Genom att använda semantiska HTML-element blir innehållet tydligare för både webbläsare och hjälpmedel [1].

CSS (Cascading Style Sheets) styr webbplatsens visuella utformning, exempelvis färger, typografi, avstånd och layout. Med CSS Grid och Flexbox kan innehåll placeras och anpassas efter tillgängligt utrymme.

JavaScript används för att skapa interaktivitet och förändra sidans innehåll utan att HTML-dokumentet behöver skrivas om. Språket kan exempelvis hantera användarhändelser, kontrollera formulär och uppdatera element på sidan.

Tillsammans gör teknikerna det möjligt att skilja mellan webbplatsens struktur, presentation och funktionalitet.

2.2 Responsiv webbdesign och Mobile First

Responsiv webbdesign innebär att en webbplats anpassar sin layout efter skärmens storlek och förutsättningar. Syftet är att innehållet ska vara lätt att läsa och använda på exempelvis mobiltelefoner, surfplattor och datorer.

Mobile First är en arbetsprincip där webbplatsens grundläggande layout utformas för mindre skärmar. Med hjälp av CSS och så kallade *media queries* kan layouten sedan förändras när skärmutrymmet ökar. CSS Grid och Flexbox används för att skapa flexibla layouter där innehållet kan placeras om utan att HTML-strukturen behöver ändras.

Genom att utgå från mobilens begränsade utrymme blir det naturligt att prioritera innehåll, navigation och användbarhet [2].

2.3 DOM och händelsehantering

DOM (Document Object Model) är en representation av HTML-dokumentet som gör det möjligt för JavaScript att läsa och ändra sidans innehåll och struktur.

Med metoder som querySelector() kan JavaScript välja ut HTML-element. Därefter kan exempelvis textinnehåll, CSS-klasser och attribut ändras. Nya element kan också skapas och läggas till på sidan.

Händelsehantering innebär att JavaScript reagerar på användarens handlingar, exempelvis klick, tangenttryckningar eller formulär som skickas. Med addEventListener() kopplas en funktion till en viss händelse.

DOM och händelsehantering används tillsammans för att skapa interaktiva funktioner utan att hela webbsidan behöver laddas om [3].

2.4 API och JSON

Ett API (Application Programming Interface) är ett gränssnitt som gör det möjligt för olika program att kommunicera och utbyta information. Webbaserade API används bland annat för att hämta data från externa tjänster.

JSON (JavaScript Object Notation) är ett textbaserat format för att lagra och överföra strukturerad information. Formatet bygger på bland annat objekt och arrayer och kan enkelt hanteras med JavaScript [4].

Med funktionen fetch() kan JavaScript läsa data från en JSON-fil eller begära information från ett webbaserat API. Informationen kan därefter användas för att skapa eller uppdatera innehåll på en webbsida [5].

Vissa API kräver en API-nyckel för att identifiera den som använder tjänsten. Om nyckeln är hemlig bör den inte placeras i JavaScript-kod som skickas till besökarens webbläsare, eftersom koden kan granskas av användaren.

2.5 Tillgänglighet och webbstandarder

Webbtillgänglighet innebär att webbplatser ska kunna användas av så många människor som möjligt, även personer med olika funktionsnedsättningar. Det handlar bland annat om tydlig struktur, läsbar text, tillräckliga färgkontraster och möjligheten att navigera med tangentbord.

W3C (World Wide Web Consortium) utvecklar webbstandarder och riktlinjer, däribland WCAG (Web Content Accessibility Guidelines). Riktlinjerna ger stöd för att skapa webbinnehåll som är mer tillgängligt [6].

Semantisk HTML, beskrivande länkar och korrekt kopplade formuläretiketter underlättar för både användare och hjälpmedel. Validering av HTML och CSS

kan dessutom användas för att upptäcka kodfel och kontrollera att webbplatsen följer språkens regler.

3 Metod

Projektet har genomförts individuellt och omfattat planering, design, utveckling, testning och publicering av webbplatsen Respawn After Bedtime.

Webbplatsen utvecklades med HTML, CSS och JavaScript enligt principen Mobile First. Arbetet byggde på kunskaper från kursens föreläsningar, tidigare laborationer och kurslitteratur. MDN Web Docs och W3Schools användes som kompletterande dokumentation.

Externa resurser användes i form av spelinformation från RAWG API, ikoner från Simple Icons och Lucide samt spelbilder från respektive spels upphovsrättsinnehavare. AI-verktyg användes som stöd vid problemlösning och framställning av visst text- och bildmaterial, men inte för direkt kodgenerering.

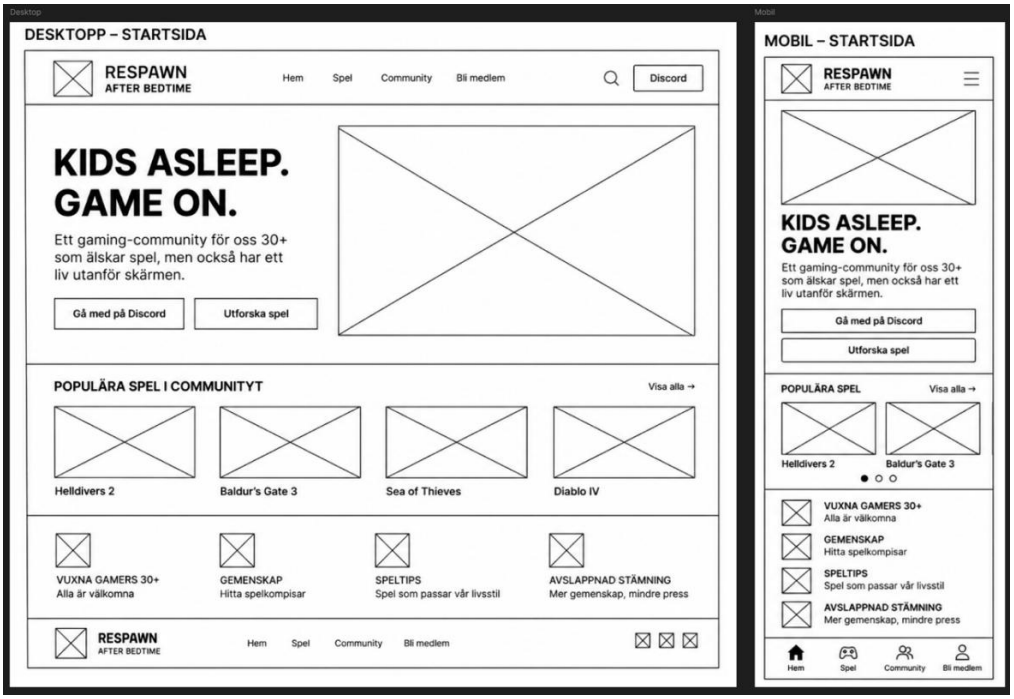
Jag ansvarade själv för utveckling, anpassning och testning av lösningarna samt de slutliga besluten om webbplatsens utformning och funktionalitet. Under arbetets gång testades webbplatsens responsivitet, funktioner och tangentbordsnavigering. HTML och CSS kontrollerades med W3C valideringsverktyg och tillgängligheten utvärderades med Google Lighthouse. Upptäckta problem åtgärdades successivt.

3.1 Planering och design

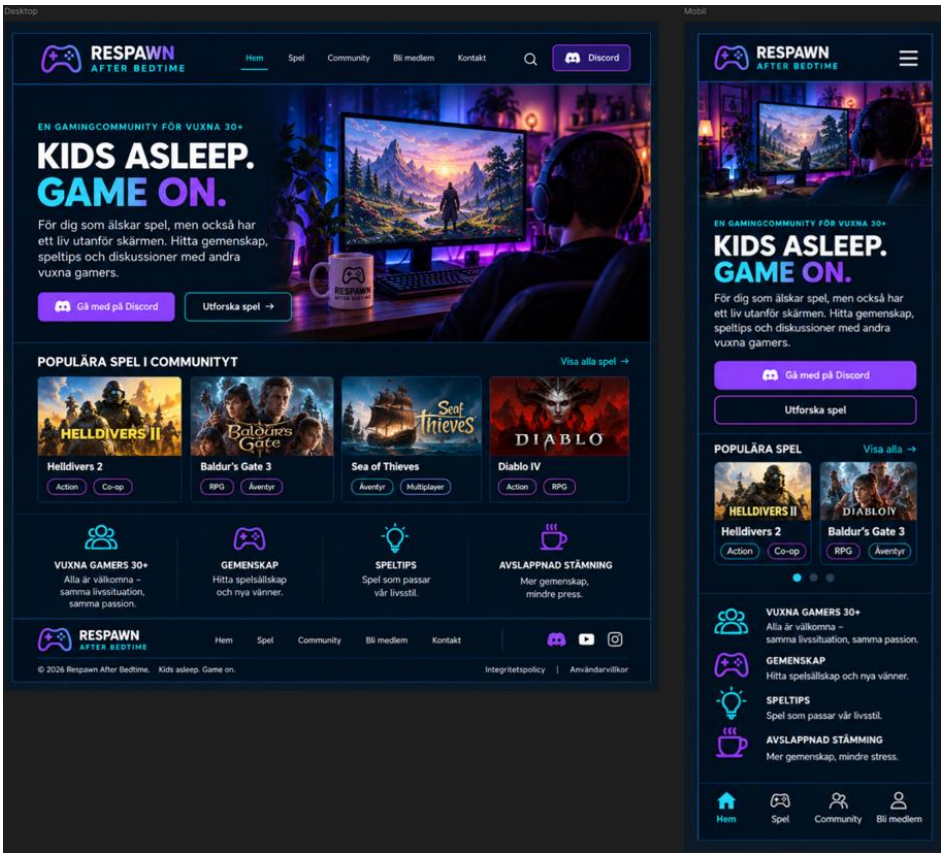
Arbetet inleddes med att identifiera webbplatsens målgrupp och planera innehållet och strukturen för webbplatsens fem sidor. Respawn After Bedtime är en fiktiv spelgemenskap, men Discord-servern som webbplatsen hänvisar till är verklig och skapades för projektet. Fokus låg på att presentera gemenskapen, visa spelinformation och göra det enkelt för besökare att ansluta via Discord.

I Figma skapades wireframes för både mobil och dator för att planera innehållets placering och navigationen. Därefter togs en grafisk design fram med en mörk bakgrund, färgaccenter i cyan och lila samt enhetlig typografi och utformning av knappar.

Designen användes som utgångspunkt under utvecklingen för att ge webbplatsens fem sidor ett enhetligt utseende.



Figur 1. Wireframes av startsidan för dator och mobil, framtagna i Figma. Egen illustration.



Figur 2. Grafisk design av startsidan för dator och mobil, framtagen i Figma. Egen illustration.

3.2 Utvecklingsmiljö och versionshantering

Webbplatsen utvecklades i Visual Studio Code med HTML, CSS och JavaScript. Projektets filer organiserades i separata mappar för bland annat stilmallar, JavaScript, bilder och spelinformation. Denna struktur gjorde det enklare att underhålla och vidareutveckla webbplatsen.

Git användes för versionshantering och GitHub för att lagra projektets kod och dokumentera ändringar genom commits. Arbetet genomfördes med separata grenar för utveckling och publicering. Den färdiga webbplatsen publicerades via GitHub Pages.

Under utvecklingen användes webbläsarens utvecklarverktyg för att kontrollera layouten vid olika skärmstorlekar och felsöka HTML, CSS och JavaScript.

3.3 Arbetsprocess och problemlösning

Utvecklingen genomfördes stegvis med utgångspunkt i designen från Figma. Först byggdes sidornas HTML-struktur och grundläggande CSS. Därefter anpassades layouten för större skärmar och funktioner med JavaScript lades till. Under arbetet testades förändringarna löpande och koden justerades när problem upptäcktes.

En utmaning var att få navigationen och innehållets layout att fungera på både mobil och dator. Detta löstes genom att kombinera CSS Grid, Flexbox och media queries. Mobilmenyn utvecklades med JavaScript och kompletterades med stöd för tangentbordsnavigering.

Ett annat problem gällde hämtningen av spelinformation från RAWG API. Den ursprungliga lösningen riskerade att exponera API-nyckeln i webbplatsens publika JavaScript-kod. Lösningen ändrades därför till ett separat Node.js-skript som hämtar spelinformationen och sparar den i en lokal JSON-fil. API-nyckeln förvaras i en .env-fil som undantas från versionshanteringen med .gitignore. Webbplatsen läser sedan spelinformationen från JSON-filen utan att behöva använda API-nyckeln.

Kontaktformuläret krävde också anpassningar för att ge tydlig återkoppling när användaren fyllde i felaktiga eller ofullständiga uppgifter. Med JavaScript infördes validering och felmeddelanden som hjälper användaren att korrigera uppgifterna.

3.4 Testning och kvalitetssäkring

Webbplatsen testades löpande under utvecklingen för att kontrollera att layout, navigation och interaktiva funktioner fungerade som avsett. Responsiviteten kontrollerades med webbläsarens utvecklarverktyg vid olika skärmbredder, inklusive mobilstorlekar ned till 320 pixlar.

Samtliga fem HTML-sidor kontrollerades med W3C HTML-validator och webbplatsens stilmall med W3C CSS-validator. Syftet var att identifiera syntaxfel och andra problem som kunde påverka webbplatsens funktion.

Tillgängligheten testades med Google Lighthouse i mobilläge. Därutöver genomfördes manuella tester av tangentbordsnavigering, mobilmeny, länkar och kontaktformulärets validering. Eventuella brister åtgärdades och funktionerna kontrollerades på nytt.

4 Konstruktion

I detta kapitel beskrivs hur webbplatsen har byggts upp, från sidornas struktur och responsiva layout till JavaScript-funktionerna och hanteringen av spelinformation från RAWG API.

4.1 Webbplatsens struktur

Webbplatsen består av fem HTML-sidor: index.html, games.html, community.html, join.html och contact.html. Sidorna är sammanlänkade genom en gemensam navigationsmeny och har samma sidhuvud, sidfot och grafiska utformning.

Projektets filer är uppdelade i mappar efter funktion. Stilmallen finns i css/styles.css, medan JavaScript-filerna ligger i mappen js. Bilder och ikoner finns i images, och spelinformationen lagras i data/games.json.

JavaScript-funktionerna är uppdelade i separata filer. main.js hanterar mobilmenyn, popular_games.js ansvarar för visningen av spelinformation och contact.js innehåller kontaktformulärets validering.

Denna uppdelning gör det möjligt att återanvända samma CSS och JavaScript på flera sidor och samtidigt hålla funktionerna åtskilda.

4.2 Responsiv layout och navigation

Webbplatsens layout utvecklades enligt principen Mobile First. Grundläggande CSS utformades för mobiltelefoner och kompletterades med media queries för större skärmar. Vid en skärmbredd på minst 769 pixlar anpassas layouten för att utnyttja det tillgängliga utrymmet.

CSS Grid används för att organisera innehåll i kolumner, bland annat för spelkort, medan Flexbox används för placering och justering av element i navigationen och andra delar av gränssnittet.

På mindre skärmar visas navigationen som en hamburgermeny. Funktionen hanteras av JavaScript i main.js, där användarens klick styr om menyn visas eller döljs. Menyn kan även stängas med Escape-tangenten, och tangentbordsfokus hanteras för att förbättra tillgängligheten.

```css
@media (min-width: 769px) {
  /* Sidhuvudets innehåll placeras på en rad */
  .site-header {
    flex-wrap: nowrap;
    gap: 24px;
    padding: 16px 4%;
  }

  /* Större logotyp på desktop */
  .logo img {
    width: 190px;
  }

  /* Hamburgermenyknappen döljs */
  .menu-toggle {
    display: none;
  }

  /* Huvudnavigationen visas alltid */
  #main-nav,
  #main-nav.open {
    display: block;
    width: auto;
  }

  /* Navigeringslänkarna placeras horisontellt */
  #main-nav ul {
    flex-direction: row;
    align-items: center;
    gap: 26px;
  }
```

Figur 3. Utdrag ur webbplatsens CSS-kod som visar hur navigationen anpassas vid en skärmbredd på minst 769 pixlar. Egen kod.

4.3 JavaScript och interaktivitet

Webbplatsens interaktiva funktioner och dynamiska innehåll hanteras med JavaScript. Koden är uppdelad i separata filer för att underlätta organisering och underhåll.

Kontaktformulärets funktionalitet finns i contact.js. När användaren försöker skicka formuläret förhindras den vanliga formulärinskickningen med prevent-Default(). JavaScript kontrollerar därefter att obligatoriska fält är ifyllda och att e-postadressen har ett giltigt format. Vid fel visas meddelanden som förklarar vad användaren behöver rätta till, och fokus flyttas till det första felaktiga fältet. Formuläret är en demonstration och skickar inga uppgifter till en server.

Spelinformationen visas dynamiskt med hjälp av popular_games.js. Skriptet använder fetch() för att läsa innehållet i data/games.json. Därefter skapas HTML-element för spelkorten med JavaScript och läggs till på sidan. På så sätt kan spelinformationen uppdateras i JSON-filen utan att HTML-koden för varje spelkort behöver ändras manuellt.

```
12  // Hämtar spelinformationen från vår lokala JSON-fil
13  async function loadPopularGames() {
14    try {
15      const response = await fetch("data/games.json");
16
17      // Kontrollerar att JSON-filen kunde hämtas
18      if (!response.ok) {
19        throw new Error("Kunde inte hämta spelinformationen.");
20      }
21
22      // Omvandlar JSON-informationen till en JavaScript-array
23      const popularGames = await response.json();
24
25      // Går igenom JSON-datan och skapar ett HTML-spelkort för varje spel
26      popularGames.forEach((game) => {
27        const card = document.createElement("article");
28        card.classList.add("game-card");
```

Figur 4. Utdrag ur popular_games.js som visar hur spelinformation hämtas från en lokal JSON-fil och hur skapandet av spelkort påbörjas. Egen kod.

4.4 RAWG API och hantering av speldata

Spelinformationen på webbplatsen kommer från RAWG API, som tillhandahåller uppgifter om datorspel [7]. För att hämta informationen utvecklades ett separat Node.js-skript, scripts/update_games.js.

Skriptet använder en API-nyckel som lagras i den lokala filen .env. Filen undantas från Git med hjälp av .gitignore och publiceras därför inte tillsammans med webbplatsens övriga filer.

När skriptet körs hämtas spelinformation från RAWG API och sparas i data/games.json. JSON-filen innehåller bland annat spelens namn, genrer, betyg och utgivningsdatum. Webbplatsens JavaScript läser sedan den lokala filen och använder informationen för att skapa spelkort.

Lösningen innebär att API-anropen görs separat från den publicerade webbplatsen. Besökarens webbläsare behöver därför inte tillgång till API-nyckeln. Spelinformationen uppdateras när Node.js-skriptet körs på nytt, inte automatiskt vid varje sidbesök.

```
10    // Hämtar API-nyckeln från miljövariablerna
11    const apiKey = process.env.RAWG_API_KEY;
```

```
45  // Hämtar spelinformation från RAWG
46  async function updateGames() {
47    if (!apiKey) {
48      throw new Error("RAWG_API_KEY saknas i miljövariablerna.");
49    }
50
51    const games = [];
52
53    for (const game of selectedGames) {
54      // Hämtar information om spelet med hjälp av dess RAWG-ID
55      const url = `https://api.rawg.io/api/games/${game.id}?key=${apiKey}`;
56      const response = await fetch(url);
57
58      if (!response.ok) {
59        throw new Error(`Kunde inte hämta ${game.name}: HTTP ${response.status}`);
60      }
61
62      // Omvandlar svaret till ett JavaScript-objekt
63      const data = await response.json();
```

Figur 5. Två kodutdrag ur update_games.js som visar hur API-nyckeln hämtas från en miljövariabel och används vid anrop till RAWG API. Egen kod.

5 Resultat

Projektet resulterade i en färdig och publicerad webbplats för Respawn After Bedtime. Webbplatsen består av fem sammanlänkade sidor och är anpassad för både mobiltelefoner och större skärmar.

Samtliga planerade huvudfunktioner har implementerats, däribland en responsiv navigationsmeny, ett kontaktformulär med validering och spelkort som skapas dynamiskt från en lokal JSON-fil. Besökare kan även ansluta till den Discord-server som skapades för projektet.

Webbplatsen har publicerats via GitHub Pages och projektets källkod finns tillgänglig i ett GitHub-repository. Genomförda tester visar att webbplatsen uppfyller de tekniska målen för responsivitet, funktionalitet och kodvalidering. Resultaten från tillgänglighetstesterna redovisas i följande avsnitt.

5.1 Måluppfyllelse

I tabell 1 redovisas projektets resultat i förhållande till de sju mål som formulerades i avsnitt 1.4.

Mål	Resultat
1. Minst tre sammanlänkade HTML-sidor	Fem sidor har utvecklats.
2. Responsiv layout med Mobile First	CSS Grid, Flexbox och media queries används.
3. Mobilmeny och formulärvalidering	Båda funktionerna har implementerats med JavaScript.
4. Dynamisk visning av spelinformation	Spelkort skapas från en lokal JSON-fil.
5. Responsivitet och tangentbordsnavigering	Funktionerna har testats på olika skärmstorlekar och med tangentbord.
6. Kodvalidering och tillgänglighetstester	W3C validatorer och Google Lighthouse har använts.
7. Publicering och versionshantering	Webbplatsen är publicerad via GitHub Pages och koden finns på GitHub.

Tabell 1. Sammanställning av projektets mål och måluppfyllelse.

5.2 Testresultat

Webbplatsen kontrollerades med W3C valideringsverktyg och Google Lighthouse. Responsivitet, tangentbordsnavigering och kontaktformulärets funktioner testades även manuellt. Resultaten sammanfattas i tabell 2.

Test	Resultat
HTML-validering, fem sidor	0 fel och 0 varningar
CSS-validering	0 fel och 3 varningar
Lighthouse – Tillgänglighet, fem sidor	100/100 på samtliga sidor
Responsivitet	Testad ned till 320 pixlars skärmbredd
Tangentbordsnavigering	Testad på webbplatsens interaktiva funktioner
Kontaktformulär	Validering och felmeddelanden testade

Tabell 2. Sammanställning av genomförda tester och testresultat.

6 Diskussion och slutsats

Projektets största utmaning var inte de enskilda teknikerna, utan att kombinera tidigare kunskaper från kursens föreläsningar och laborationer i en sammanhängande webbplats. Teknikerna hade behandlats var för sig under kursen, men i projektet behövde de fungera tillsammans på fem sidor med en gemensam design och navigation.

En viktig del av arbetet var att omsätta planeringen i Figma till en fungerande webbplats. Genom att tidigt bestämma färger, typografi och gränssnittselement kunde en konsekvent utformning behållas under utvecklingen. Arbetet med Mobile First innebar samtidigt att layouten behövde anpassas efter olika skärmstorlekar utan att webbplatsens funktionalitet försämrades.

Den mest betydande tekniska förändringen gällde hanteringen av spelinformation från RAWG API. Den ursprungliga lösningen riskerade att exponera API-nyckeln i publik JavaScript-kod. Genom att i stället använda ett separat Node.js-skript och en lokal JSON-fil kunde spelinformationen visas utan att besökarens webbläsare behövde tillgång till nyckeln. En nackdel är att informationen inte uppdateras automatiskt, vilket skulle kunna vidareutvecklas i framtiden.

Testresultaten visar att webbplatsen klarade HTML-valideringen utan fel eller varningar och fick 100 av 100 poäng i Lighthouse-kategorin Tillgänglighet på samtliga fem sidor. Resultaten ger stöd för att de tekniska lösningarna fungerar väl, men automatiserade tester kan inte ensamma garantera fullständig tillgänglighet. Därför genomfördes även manuella tester.

En begränsning är att kontaktformuläret endast demonstrerar validering och inte skickar några meddelanden. Vid en framtida vidareutveckling skulle formuläret kunna kopplas till en tjänst för meddelandehantering. Även hämtningen av spelinformation skulle kunna automatiseras.

Sammanfattningsvis har projektet framför allt gett erfarenhet av att planera, genomföra och färdigställa ett större webbutvecklingsarbete. Det har visat hur tidigare inlärda tekniker kan kombineras med nya lösningar, samt betydelsen av att kunna ompröva tekniska beslut när problem upptäcks.

7 Källförteckning

[1] MDN Web Docs. *Structuring content with HTML* [Internet]. Mozilla; [hämtad 2026-10-27]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content

[2] MDN Web Docs. *Responsive web design* [Internet]. Mozilla; [hämtad 2026-10-27]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

[3] MDN Web Docs. *Document Object Model (DOM)* [Internet]. Mozilla; [hämtad 2026-10-27]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

[4] MDN Web Docs. *Working with JSON* [Internet]. Mozilla; [hämtad 2026-10-27]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON

[5] MDN Web Docs. *Using the Fetch API* [Internet]. Mozilla; [hämtad 2026-10-27]. Tillgänglig från: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

[6] World Wide Web Consortium (W3C). *Web Content Accessibility Guidelines (WCAG) 2.2* [Internet]. 2024 [hämtad 2026-10-27]. Tillgänglig från: https://www.w3.org/TR/WCAG22/

[7] RAWG. *Explore RAWG Video Games Database API* [Internet]. [hämtad 2026-10-27]. Tillgänglig från: https://rawg.io/apidocs